

Candidate: Prince Mehra

Email: princemehra3666@gmail.com

GitHub:

https://github.com/jeorgeiiii/remoteward_feedback_app



RemoteWard Feedback Application

Design Choices -

The application was built using the Flutter framework following a clean layered architecture to ensure separation of concerns, testability, and maintainability.

State Management (BLoC):

I chose the BLoC (Business Logic Component) pattern using the flutter_bloc package because it enforces a clear unidirectional data flow – UI sends events, BLoC processes them, and UI reacts to state changes. This keeps business logic completely separate from UI code, making the app predictable and easy to debug. The app uses three focused BLoCs:

- AuthBloc – manages Google Sign-In and authentication state.
- FeedbackBloc – maintains a draft feedback entry across the three collection screens.
- ExportBloc – handles the biometric gate and CSV/PDF export flow

Dependency Injection (get_it):

All services, repositories, and BLoCs are registered in a central injection_container.dart file using get_it. This decouples component creation from usage, improves testability, and makes the codebase easier to maintain.

Database Layer (sqflite):

A dedicated DatabaseService class handles all SQLite operations. The FeedbackRepository acts as the single coordinator between BLoCs and the database, ensuring that no UI or BLoC code directly touches SQLite. This respects the requirement for a "dedicated database service layer."

Scoped Storage (MediaStore):

Media files (images) and exported CSV/PDF files are stored in the device's Downloads folder using the media_store_plus package, which wraps Android's MediaStore API. This ensures compliance with Android 10+ scoped storage requirements. Images are first copied to the app's private documents directory for reliable access at export time, then mirrored to Downloads.

Biometric Authentication:

The export function is secured by local_auth, which supports fingerprint, PIN, pattern, or password – exactly as required. The app also provides a clear warning when no screen lock is set up on the device.

PDF Export (Bonus):

In addition to the required CSV export, I added a PDF export option with embedded images. This makes the feedback report more professional and readable on any device, and addresses the "can't open CSV on emulator" issue by providing an in-app preview using the printing package.



Challenges Faced -

Google Sign-In 7.x Migration:

The biggest challenge was integrating google_sign_in version 7.x, which introduced a breaking change – a new singleton-based API with initialize() and separate authenticate()/authorize() steps. Most existing tutorials and code generators still use the old 6.x flow (GoogleSignIn().signIn()), which would not compile. I had to carefully study the official documentation and implement the correct 7.x pattern to make authentication work.

SHA-256 Fingerprint Issue on Physical Device (Biggest Issue):

Google Sign-In worked perfectly on the emulator but failed silently on a physical Vivo phone – the account picker would not appear, and the app would just spin endlessly. After extensive debugging, I discovered that Android's modern Credential Manager requires the SHA-256 fingerprint to be present in the google-services.json file. The Firebase console showed both SHA-1 and SHA-256 fingerprints, but the downloaded config file only contained SHA-1 – a known Firebase configuration cache issue (Unfortunately this error hasn't been fixed by dev since 2020's).

Instead of fighting with manual JSON edits or Firebase's caching behavior, I solved this by switching from the authenticate() method to the signIn() method on the GoogleSignIn instance. This forces the older, web-based OAuth flow, which does not validate the app's SHA-256 fingerprint at all. This approach is reliable, requires no manual file editing, and works consistently on all physical devices and emulators, regardless of the SHA configuration. The authentication still passes the ID token to Firebase Auth, so all downstream functionality – user state, allowlist checks, and export flows – remains unchanged.

MediaStore API Differences:

The media_store_plus package's saveFile API varies slightly across versions, which caused some runtime uncertainty. I isolated all MediaStore operations in the MediaStorageService class, making it a one-file fix if adjustments are needed for different package versions.

Image Embedding in PDF (Took a little time but fixed it):

Initially, the PDF only showed media filenames instead of embedding the actual images. This was because the saved media paths were not permanent. I fixed this by copying picked images into the app's private documents/feedback_media/ directory immediately, ensuring the PDF could read the files at export time.



Potential Improvements (If Given More Time) -

Multi-Owner Support / Admin Panel:

Currently, only one device owner can use the app. With more time, I would build a cloud backend (Firebase Firestore) to sync feedback across multiple devices and create a web dashboard where owners can view, filter, and export all feedback centrally.

Firebase Storage for Media:

Instead of storing media only locally on the device, I would upload images/videos to Firebase Storage and store the download URLs in the database. This would allow media to be accessed from any device and prevent data loss if the app is uninstalled.

Offline Sync & Retry:

Add offline persistence with sync queues – if the device is offline when submitting feedback, store it locally and retry upload when connectivity is restored.

User Management (Allowlist with Role-Based Access):

I already implemented an in-app email allowlist (info@remoteward.com and the owner's email). With more time, I would move this to a Firestore collection, allowing admins to add/remove approved users dynamically without updating the app.

Improved UI/UX:

Add dark mode support, more animations, and a richer media preview (e.g., thumbnail gallery instead of text links). The current UI is clean but could be enhanced with more feedback cues.

Unit and Integration Tests:

While I have some BLoC unit tests, I would expand test coverage to include integration tests for the full flow → sign-in → collect feedback → export → using flutter_driver or integration_test.

Push Notifications:

Notify the device owner when new feedback is submitted, even if the app is closed – useful for real-time monitoring.

Conclusion -

All functional requirements of the assignment have been met:

- Google Sign-In with Firebase Authentication
- BLoC state management with flutter_bloc
- SQLite persistence with dedicated service layer
- Dependency Injection with get_it
- Scoped storage via MediaStore (Downloads folder)
- Five screens with smooth transitions
- Biometric-gated CSV export
- Bonus: PDF export with embedded images

The app is production-ready and successfully tested on both emulator and physical Android devices (Vivo V23 Pro). The source code is available on GitHub at the link above.